

Week 1 Urban Data Platform Design Report

The platform integrates four heterogeneous urban datasets into Delta tables using reusable Spark ingestion and explicit domain transformations. Its analytical output retains one row per accepted taxi trip with weather, air quality and both endpoint locations. Monthly partitions are the configured default for taxi facts; small contextual tables remain unpartitioned. The design prioritizes reproducible normalization, visible missing context and verifiable trip preservation.

Data catalog

Dataset	Entity and operational key	Join and temporal fields
Taxi Parquet	One trip; hash of vendor, UTC endpoints, location IDs, fare and distance	Endpoint location IDs; UTC endpoint hours; local timestamps and pickup calendar
Weather CSV	One hourly observation; source year/month/day/hour	UTC observation_datetime; source calendar and UTC observation_date
Air quality CSV	One monitor occurrence/hour; hash of state, county, site, POC and UTC time	Derived borough and UTC observation time, date and hour
Zone CSV	One reference entry; location_id	Both trip location IDs; no temporal source field

Taxi categories include vendor, payment type, rate code, store-and-forward flag and locations. Weather uses condition codes, cloud cover and source flags. Air uses county, borough, instrument, parameter and units. Zones contain borough, zone and service-zone labels. Taxi growth is monthly trip arrivals; context growth is new hours and monitors; zone changes are occasional reference updates.

Taxi has no guaranteed natural unique key. Its SHA-256 surrogate defines a duplicate policy, can collapse indistinguishable trips and changes when fare or distance is corrected. The weather key assumes one source series. The air key assumes the supplied single PM2.5 parameter; multi-station weather or multi-pollutant air requires broader keys. These are snapshot identities, not established correction-stable identifiers.

Scope and observed data

Taxi files cover January-March 2024. Weather contains 8,784 hourly rows. Nationwide air input is filtered to New York State and the five NYC counties for Silver; Bronze preserves the national snapshot. Current run counts and coverage are recorded in the execution JSON and the companion validation record; the benchmark report names its source artifact.

This document describes the current source and configuration. The companion task answers provide the full transformation inventory and every Week 1 discussion answer. The benchmark report identifies the saved numerical evidence and its limits.

Storage architecture

Original files remain in `data/`. Four Bronze Delta tables hold source snapshots, and five Silver tables hold cleaned sources and integrated `_taxi_trips`. Delta supplies a common read/write interface and table-level transaction semantics. Tables are addressed by filesystem path; a separately deployed catalog is not required.

Path below storage	Purpose and write behavior
<code>delta/bronze/raw_*</code>	Four source tables; overwrite snapshots
<code>delta/silver/clean_*</code>	Four validated tables; overwrite
<code>delta/silver/integrated_taxi_trips</code>	Enriched trip facts; overwrite after identity checks
<code>delta/quarantine/<dataset></code>	Rejections and reasons; append separately per dataset
<code>delta/metadata/ingestion_runs</code>	Successful ingestion audit rows; append
<code>delta/benchmark/<strategy>/<run_id></code>	Fresh experiment paths preserving earlier runs
<code>metrics</code>	Latest and timestamped JSON summaries

Bronze sanitizes source column names and adds an ingestion timestamp. It preserves fields, not file bytes; original files provide the byte-level source. Separate quarantine tables accommodate heterogeneous schemas and retain rejection reason, run ID, version and time. Additive schema merging is allowed there, but incompatible type changes need explicit migration.

Partitioning and lookup roles

The active YAML partitions taxi and integrated tables by `pickup_year` and `pickup_month`, based on New York local pickup time. Weather, NYC air, zones and Bronze are unpartitioned. Direct constructor defaults still use daily taxi/integration partitions; the normal runner explicitly supplies YAML and therefore uses monthly partitions.

Zones are spatial dimensions. Weather and borough/citywide hourly air aggregates serve as context lookups; raw monitor readings are observation facts. Small clean context tables favor broadcasting and do not justify fine partitions. High-cardinality or sparse partitions can increase metadata, file-open and scheduling costs. Coarse partitions can read excess data or constrain parallel writing. Predicates must be compatible with partition columns; a date filter alone does not prove pruning of year/month partitions.

At twentyfold scale, distinguish more trips per day from a longer date range. Reassess daily versus monthly layouts, compaction, target file sizes, worker resources and multiple writers per large partition. The integrated writer redistributes by partition keys, which can limit parallelism over a short time range. Recheck broadcasts as sources grow. Shared object storage, a catalog and incremental scheduling are proposed extensions, not demonstrated local features.

Common data model

Silver uses lower snake_case, typed UTC timestamps and unit-bearing measurement names where defined. UTC instants can be exchanged as YYYY-MM-DDTHH:mm:ssZ; Delta stores TimestampType values. Taxi local timestamps and calendar columns support local-day analysis. Context observation_date uses UTC, which differs from local pickup_date near midnight.

Domain	Normalization and derived attributes
Taxi	Semantic renames, configured integer IDs/double measures, local-to-UTC endpoints, local calendar, rounded duration and trip hash
Weather	Calendar to UTC hour, observation key/date and standard weather measurement names
Air	GMT date/time to UTC, NYC scope, county-to-borough mapping, site/POC identity and concentration fields
Zones	Integer location_id and trimmed borough, zone and service_zone

Taxi wall time is interpreted as America/New_York. A UTC-to-local round-trip check rejects nonexistent spring-transition times. Ambiguous autumn times follow Spark resolution; Q1 does not test that case. Air uses EPA GMT fields rather than treating its local standard-time fields as daylight time. Weather defaults to UTC because the CSV has no timezone metadata; export settings must confirm this assumption.

Configured IDs and calendar values are integer, measures and money double, labels/hash IDs string, dates DateType and match flags boolean. Unconfigured taxi attributes retain source types. No general unit conversion is applied. Decimal monetary values and explicit concentration-unit validation would strengthen the model for production use.

Missing values and quality rules

Missing passenger counts default to one, and missing tips/tolls to zero. These assumptions can bias aggregates. Malformed non-null casts still cause rejection, even if defaults are supplied later. Missing weather and precipitation remain null; missing endpoint labels become Unknown. pickup_pm25_borough retains a nullable borough-hour estimate; pickup_pm25_citywide retains the available-site city mean for the same UTC hour. Both are double concentrations in the source PM2.5 units. Corresponding dropoff columns use the dropoff hour.

Taxi rules check endpoint times, rounded duration >0.1 and <=1440 minutes, Q1 2024 local pickup, fare 0-5000, distance 0-500, passengers 0-10, finite nonnegative total/tips/tolls and present endpoint IDs. Weather checks timestamp/calendar, temperature -40 to 60, humidity 0-100, finite nonnegative wind/observed precipitation, finite positive observed pressure, cloud cover 0-8 and condition codes 1-27. Air checks UTC year 2024, PM2.5 0-1000 and monitor/geography fields. Zones require key and nonblank borough. All datasets reject configured cast failures, null keys and duplicate occurrences.

The framework does not validate all optional attributes, all reference relationships or all units. YAML lists every field mapping and rule; Task 4 in the companion answers documents projections, defaults, derived values and these limitations. Schema changes should update that inventory, version and focused regression fixtures together.

Reusable ingestion framework

BaseDatasetIngestor provides source loading, required-column checks, Bronze persistence, renaming/casts, scope selection, domain normalization, quality rules, deduplication, quarantine, Silver persistence and metadata. Four subclasses implement source-specific semantics. `src/common/config.py` validates YAML and constructs ingestors through a registry. All source, rule and integration schemas resolve before table writes. Explicit constructor paths and isolated fixture audits prevent accidental writes to default tables.

CSV is read by header as strings, avoiding silent positional coercion when columns move. Missing required columns fail explicitly; extra columns are allowed. Structural CSV corruption uses FAILFAST and can stop the run. Tolerant field casts record `invalid_type` errors for quarantine. Naming collisions fail instead of silently replacing a column.

Named SQL validity predicates come from `datasets.validation`, with thresholds from `quality_rules` and configured source timezones. False or null means invalid. Key fields must be non-null. Valid rows are ranked deterministically within key groups; one survives and later occurrences are quarantined as `duplicate_key`. This is snapshot deduplication, not correction resolution.

Metadata and execution controls

Every successful ingestion asserts `initial_records = valid_records + rejected_records + out_of_scope_records`. Rejected counts include duplicates; invalid counts exclude them. Geographically excluded air rows are counted separately from invalid NYC rows. Completed ingestions append counts, duration, schema version, run ID, paths, timezone, partitions and completion time to Delta metadata. Duration excludes that final audit write.

The runner saves timestamped and atomically replaced latest JSON with runtime, configuration, status and stage results. Ingestion stages share one execution ID. Artifact failures fail the command while preserving earlier processing errors. Raw and classified rows are materialized once for reuse. Failed executions can appear in JSON; successful-ingestion audit rows are not comprehensive failure monitoring. Schema versions are maintained manually. Repeated runs overwrite Bronze/Silver but append quarantine and audit history. Delta commits are atomic per table; there is no single transaction across all stages, so a failure can leave tables from different refresh stages.

Engineering decisions and extension cost

Configuration separates mappings, thresholds and paths from reusable control flow. Python subclasses retain semantic transformations such as timezone interpretation, monitor identity and borough mapping. That division avoids duplicated ingestion code without pretending domain rules can be inferred from a schema.

Adding twenty datasets requires source specifications, registry entries, fixtures and domain transformations. The base class receives an explicit specification; the current runner validates the registered Week 1 dataset set before processing. New formats need readers. Integration relationships, schema compatibility, scheduling and resource allocation require explicit design. Incremental ingestion, historical dimensions and full monitoring remain later extensions.

Integration strategy

The pipeline loads the four clean tables and left-joins zone labels twice by endpoint location ID. It truncates UTC endpoint times to the containing hour and joins weather on exact observation_datetime. A January pickup at 08:37 New York matches 13:00 UTC. Context is assumed to represent its labelled hour; there is no nearest-hour search, interpolation or forward fill.

Air processing first averages instruments within site/hour, then gives each site equal weight in borough/hour and city/hour means, rounded to two decimals. Two parallel left joins retain pickup_pm25_borough and pickup_pm25_citywide. Missing borough values stay null; every trip in the same UTC hour receives the same citywide value, null if no sites report. The convenience column pickup_pm25 still prefers borough, then citywide, then null, with pickup_pm25_source recording its origin. Dropoff retains the same separate estimates and fallback.

Small reference and aggregate DataFrames are broadcast. Missing labels become Unknown. Weather-match booleans, precipitation-missing booleans and PM2.5 source labels (borough, citywide, missing) expose coverage. Missing context does not remove accepted taxi trips.

Correctness and analytical limitations

Before publication, source and output must have unique non-null trip IDs. Anti-joins in both directions enforce exact ID-set equality, detecting losses, fanout and substitutions. These checks and output materialization cost compute and storage but protect the declared one-trip grain.

The supplied Q1 PM2.5 data has usable readings in the Bronx, Brooklyn and Queens, but none in Manhattan or Staten Island. The citywide estimate represents available sites, not equal coverage of five boroughs. The historical validation report indicates 90.1% of pickups lacked a matching borough-hour estimate; this is not a new coverage measurement. Use the citywide series for citywide demand analysis and borough estimates with explicit missing-coverage reporting for borough comparisons. Neither is route-level exposure or causal evidence. Weather matching does not confirm its assumed timezone.

Benchmark informed trade offs

The companion benchmark report is generated directly from one successful raw-taxi metrics artifact. It contains layout ingestion time, storage size, file counts and five query comparisons with sample variability. Monthly partitioning remains provisional; selective and full-range workloads can favor different layouts.

Each layout independently reads and cleans raw taxi input, excluding shared Bronze/quarantine/audit writes. Warm repeated queries and one fixed-order ingestion trial limit causal and scalability conclusions. Saved JSON retains plans, samples and effective configuration; directory partitioning and writer distribution are separate choices.

Sources: assignment_full_text.txt, Week 1; config/platform_config.yaml; src/ingestion/; src/integration/pipeline.py; src/storage/benchmark.py; scripts/run_week1.py; reports/week1/benchmark_report.md and its named metrics artifact.